



Universidad autónoma de baja california

Ingeniería en computación

Internet de las cosas

Lab-taller 7: SMS

Aguilar Noriega Leocundo

Garcia Chaves erik

viernes 06 de marzo del 2026

- **Objetivo:**

Al detectar un evento, como puede ser el que se presionó un botón o se superó el umbral de la medición de un ADC se envié un mensaje por SMS

- Envío de mensaje SMS

- 1- Integre a la aplicación ESP-IDF la detección de un evento puede ser el caso de un interruptor o un umbral del ADC
- 2- Una vez que se haya detectado el evento, se debe de incluir la información para envío de mensaje SMS con el siguiente formato:

https://tj-uabc.net/sms/snd.php?info=<usuario>:<num_tel>:<mensaje>

Donde:

<usuario>: hace referencia a la matricula

<num_tel>: número de teléfono a la que enviara el mensaje

<mensaje>: pequeño mensaje de texto correspondiente a la notificación a enviar con un máximo de 32 caracteres

- **Procedimiento:**

Para realizar el envío de mensaje el molde del coigo es practicamente el mismo que se uso para telegram, solo con los cambios necesarios para enviar el mensaje mediante un metodo GET y la funcion que envia el mensaje se construye el mensaje mas facil puesto que se sabe de ante mano que es lo que va a completar el cuerpo de la peticion.

```

void https_sendMessage_perform_post() {
    char url[512];
    char info_raw[256];

    char *user = "01275863";
    char *tel = "6611366387";
    char *message = "prueba_ESP_E";

    snprintf(info_raw, sizeof(info_raw), "%s:%s", user, tel, message);

    snprintf(url, sizeof(url), "https://tj-uabc.net/sms/snd.php?info=%s", info_raw);

    ESP_LOGI(TAG, "URL armado https: %s", url);

    esp_http_client_config_t config = {
        .url = url,
        .transport_type = HTTP_METHOD_GET,
        .event_handler = _http_event_handler,
        .cert_bundle_attach = esp_cert_bundle_attach,
    };

    ESP_LOGW(TAG4, "Iniciando cliente HTTP");
    esp_http_client_handle_t client = esp_http_client_init(&config);

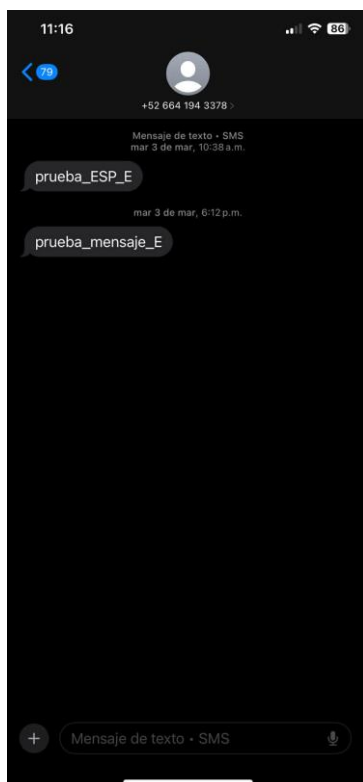
    esp_err_t err = esp_http_client_perform(client);
    if (err == ESP_OK) {
        ESP_LOGI(TAG3, "HTTP POST Status = %d, content_length = %d", esp_http_client_get_status_code(client), esp_http_client_get_content_length(client));
    } else {
        ESP_LOGE(TAG3, "HTTP POST failed: %s", esp_err_to_name(err));
    }

    esp_http_client_close(client);
    esp_http_client_cleanup(client);
    ESP_LOGI(TAG3, "Heap libre: %d bytes", esp_get_free_heap_size());
}

```

La parte que completa el mensaje con las 3 variables iniciales que es la matrícula **1275863**, el numero: **6611366387** y el mensaje que en este caso seria **“prueba_ESP_E”**

Cuando el programa se ejecuta y se recibe el evento, el programa envía al servidor el metodo GET que a su vez se reenvía a un repetidor que con red de celular para poder enviar el mensaje al número enviado.



Conclusiones:

Durante el desarrollo de la aplicación pudimos ponder a prueba otro metodo para poder solicitar funciones, en otras practicas pediamos de un servidor un servicio, tomabamos la hora, con telegram y SMS lo que hicimos ahora seria enviar desde nuestros dispositivos ESP32 hacia los distintos servicios que esta pasando en el mundo real, por lo que ahora lo que seguiria seria una interaccion entre un servicio de mensajeria y nuestros dispositivos ESP, seria una aplicacion completa en cunado IoT de la actualidad.